

Notación UML de Clases

Estructura, visibilidad y relaciones entre clases, con ejemplos en Python

M^a Dolores Rodríguez Moreno



Universidad
de Alcalá



ISG

Índice

- ¿Qué es un diagrama de clases?
- Anatomía de una clase: nombre, atributos y operaciones
- Visibilidad de los miembros
- Sintaxis de atributos y operaciones
- Relaciones entre clases (I): asociación, agregación, composición
- Relaciones entre clases (II): generalización, realización, dependencia
- Multiplicidad
- Resumen

¿Qué es un diagrama de clases?

- Diagrama estructural de UML que modela la organización estática de un sistema orientado a objetos.
- Representa las clases del sistema junto con sus atributos y operaciones.
- Muestra las relaciones entre esas clases (asociación, herencia, etc.).
- Es el diagrama UML más utilizado en el análisis y diseño de software.



Cada caja representa una clase; las líneas entre cajas representan cómo se relacionan entre sí.

Anatomía de una clase

Persona
- nombre : str - edad : int # activo : bool
+ Persona(nombre : str, edad : int) + caminar() + saludar(otra : Persona) : str

1. Nombre de la clase

Primer compartimento. Va centrado y en negrita.

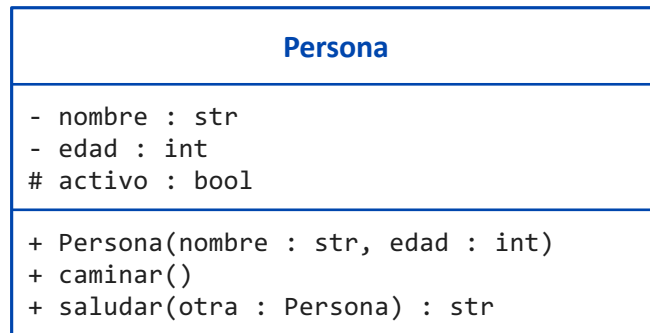
2. Atributos

Segundo compartimento. Cada línea: visibilidad nombre : tipo.

3. Operaciones (métodos)

Tercer compartimento. Cada línea: visibilidad nombre(parámetros) : tipoRetorno.

Ejemplo: una clase en Python



```
class Persona:
    def __init__(self, nombre: str, edad: int):
        self.__nombre = nombre        # - nombre : str
        self.__edad = edad            # - edad : int
        self._activo = True           # # activo : bool

    def caminar(self) -> None:         # + caminar()
        print(f"{self.__nombre} camina")

    def saludar(self, otra: "Persona") -> str:
        return f"Hola, {otra.__nombre}"
```

En UML el constructor lleva el nombre de la clase; en Python es `__init__`, y los atributos se crean dentro de él.

Visibilidad de los miembros



Público

Accesible desde cualquier otra clase del sistema.



Privado

Accesible sólo desde dentro de la propia clase.



Protegido

Accesible desde la propia clase y desde sus subclases.



Paquete

Accesible desde las clases del mismo paquete.

El símbolo precede siempre al nombre del atributo o de la operación. Los miembros estáticos se representan subrayados.

Ejemplo: visibilidad en Python



titular

Público: nombre sin prefijo



_saldo

Protegido: un guion bajo (convención)



__pin

Privado: dos guiones bajos (name mangling)



-

Paquete: no tiene equivalente en Python

```
class Cuenta:
    def __init__(self, titular: str, pin: str):
        self.titular = titular      # + público
        self._saldo = 0.0          # # protegido
        self.__pin = pin           # - privado

c = Cuenta("Ana", "1234")
print(c.titular)                  # Ana
print(c._saldo)                   # 0.0 -> funciona, pero no se debe
print(c.__pin)                    # AttributeError
print(c._Cuenta__pin)            # 1234 -> sigue siendo accesible
```

Python no impone la visibilidad: es un acuerdo entre programadores, no una barrera del lenguaje.

Sintaxis de atributos y operaciones

Atributos

visibilidad nombre : tipo [multiplicidad] = valorInicial

- saldo : float = 0.0

Sólo el nombre es obligatorio; el resto de elementos son opcionales.

Operaciones

visibilidad nombre(par : tipo, ...) : tipoDeRetorno

+ ingresar(cantidad : float) : bool

El tipo de retorno se omite cuando la operación no devuelve nada.

Una clase abstracta se escribe con el nombre en cursiva; las operaciones abstractas, también.



Ejemplo: de UML a type hints

UML

- saldo : float = 0.0

+ ingresar(cantidad : float) : bool

+ cerrar()

+ total : int = 0

Python

`self.__saldo: float = 0.0`

`def ingresar(self, cantidad: float) -> bool:`

`def cerrar(self) -> None:`

`total: int = 0    # en el cuerpo de la clase`

Los type hints (PEP 484) traducen casi literalmente la notación UML. Lo subrayado (estático) es un atributo de clase.

Relaciones (I): el todo y sus partes



Asociación

Línea continua. Vínculo estructural entre dos clases con ciclos de vida independientes.



Agregación

Rombo hueco en el lado del todo. La parte puede existir sin el todo.

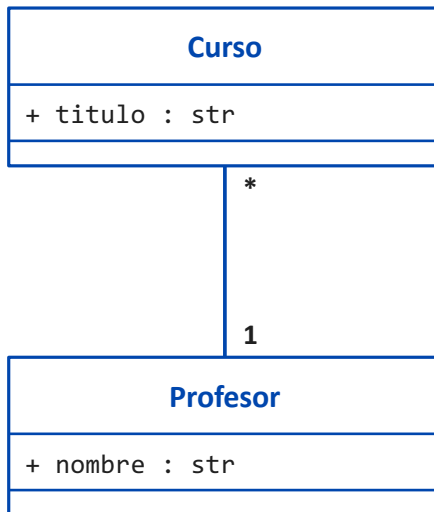


Composición

Rombo relleno en el lado del todo. La parte no sobrevive a la destrucción del todo.

Agregación y composición son formas especializadas de asociación: ambas expresan una relación todo-parte.

Ejemplo: asociación



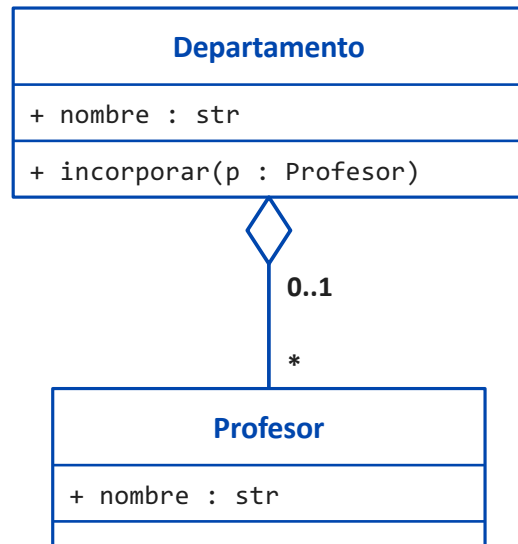
```
class Profesor:
    def __init__(self, nombre: str):
        self.nombre = nombre

class Curso:
    def __init__(self, titulo: str, profesor: Profesor):
        self.titulo = titulo
        self.profesor = profesor    # la asociación

marta = Profesor("Marta")
curso = Curso("Ingeniería del Software", marta)
del curso    # marta sigue existiendo
```

El atributo profesor no se repite dentro de la caja: lo representa la línea. Cada curso tiene 1 profesor; un profesor, varios cursos.

Ejemplo: agregación



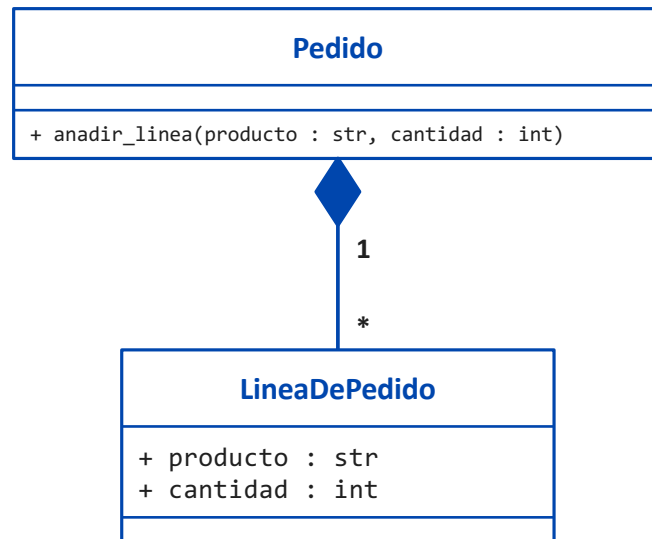
```
class Departamento:
    def __init__(self, nombre: str):
        self.nombre = nombre
        self.profesores: list[Profesor] = []

    def incorporar(self, p: Profesor) -> None:
        self.profesores.append(p) # Lo recibe ya creado

luis = Profesor("Luis") # nace fuera
dep = Departamento("Computación")
dep.incorporar(luis)
del dep # luis sigue existiendo
```

Clave de la agregación: la parte se crea fuera y se recibe como parámetro. Su vida no depende del todo.

Ejemplo: composición



```
class LineaDePedido:
    def __init__(self, producto: str, cantidad: int):
        self.producto = producto
        self.cantidad = cantidad

class Pedido:
    def __init__(self):
        self.__lineas: list[LineaDePedido] = []

    def anadir_linea(self, producto: str,
                    cantidad: int) -> None:
        # el propio pedido crea la línea
        linea = LineaDePedido(producto, cantidad)
        self.__lineas.append(linea)
```

Clave de la composición: el todo crea sus partes y no las expone. Comparad `anadir_linea` (crea) con `incorporar` (recibe).

Relaciones (II): herencia y uso



Generalización

Línea continua con triángulo hueco hacia la superclase. Relación «es un».



Realización

Línea discontinua con triángulo hueco. Una clase implementa una interfaz.

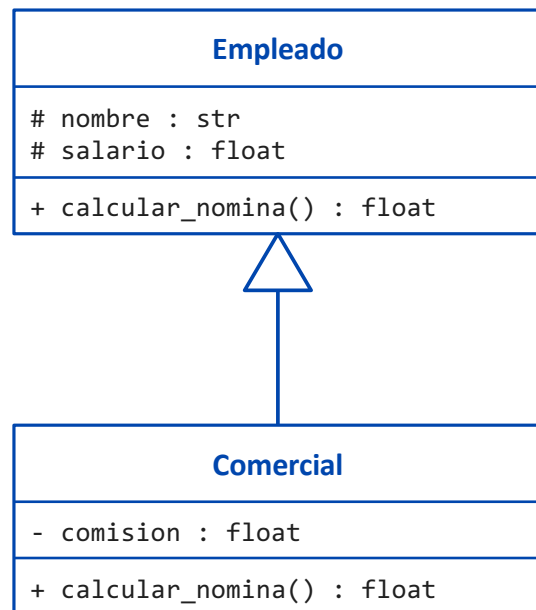


Dependencia

Línea discontinua con punta de flecha abierta. Uso puntual, sin atributo asociado.

La flecha apunta siempre hacia el elemento más general o hacia el elemento del que se depende.

Ejemplo: generalización



```
class Empleado:
    def __init__(self, nombre: str, salario: float):
        self._nombre = nombre
        self._salario = salario

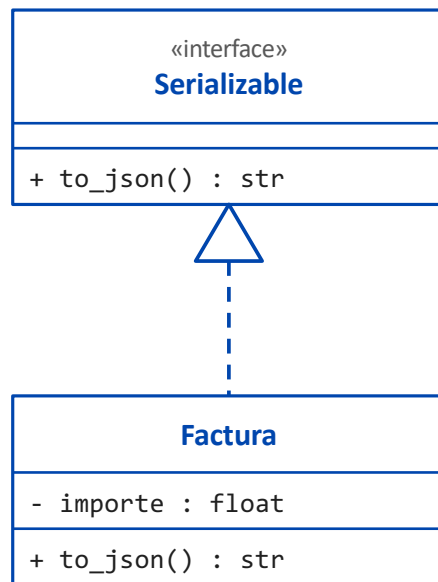
    def calcular_nomina(self) -> float:
        return self._salario

class Comercial(Empleado):          # Comercial es un Empleado
    def __init__(self, nombre: str, salario: float,
                  comision: float):
        super().__init__(nombre, salario)
        self._comision = comision

    def calcular_nomina(self) -> float:
        return super().calcular_nomina() + self._comision
```

La herencia se expresa entre paréntesis: `class Comercial(Empleado)`. Repetir `calcular_nomina` en la subclase indica que se redefine.

Ejemplo: realización



```
from abc import ABC, abstractmethod

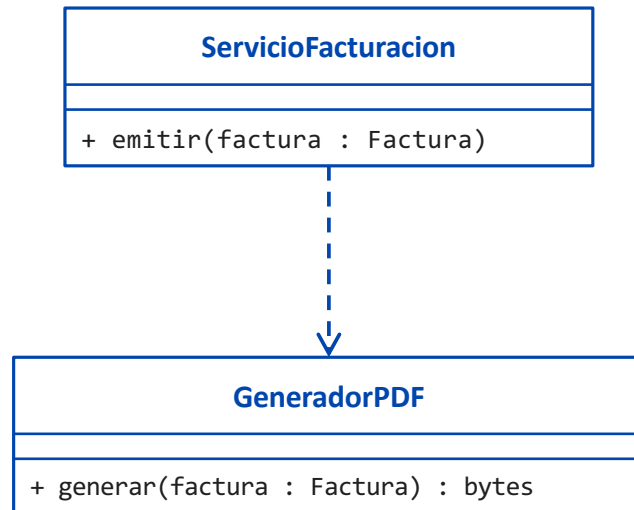
class Serializable(ABC):          # «interface»
    @abstractmethod
    def to_json(self) -> str: ...

class Factura(Serializable):     # realiza Serializable
    def __init__(self, importe: float):
        self.__importe = importe

    def to_json(self) -> str:
        return f'{"importe": {self.__importe}}'
```

Python no tiene la palabra interface: se modela con una clase abstracta (ABC) o con typing.Protocol.

Ejemplo: dependencia



```
class GeneradorPDF:
    def generar(self, factura: Factura) -> bytes:
        return factura.to_json().encode() # simplificado

class ServicioFacturacion:
    def emitir(self, factura: Factura) -> None:
        generador = GeneradorPDF() # variable local
        pdf = generador.generar(factura)
        # al terminar el método, la relación desaparece
```

Si el servicio guardara el generador en `self.generador`, dejaría de ser una dependencia y pasaría a ser una asociación.

Multiplicidad



Un pedido contiene una o más líneas; cada línea pertenece a un único pedido.

1 exactamente uno

0..1 cero o uno (opcional)

***** cero o más (igual que 0..*)

1..* uno o más

2..5 rango concreto

4 número fijo de instancias

Ejemplo: multiplicidad en Python

1	profesor: Profesor
0..1	tutor: Profesor None
*	alumnos: list[Alumno]
1..*	list[...] + validación

```
class Alumno:
    def __init__(self, nombre: str):
        self.nombre = nombre
        self.tutor: Profesor | None = None      # 0..1

class Grupo:
    def __init__(self, profesor: Profesor):
        self.profesor = profesor                # 1
        self.alumnos: list[Alumno] = []         # *

class Pedido:
    def __init__(self, lineas: list[LineaDePedido]):
        if not lineas:                          # 1..*
            raise ValueError("Al menos una línea")
        self.__lineas = lineas
```

El tipo indica si hay uno o muchos; los mínimos (el 1 de 1..) se comprueban en código. X | None requiere Python 3.10+.*

Resumen

Tres compartimentos

Nombre, atributos y operaciones. Sólo el nombre es obligatorio.

Cuatro visibilidades

Público, privado, protegido y de paquete, siempre delante del nombre.

Seis relaciones

Asociación, agregación, composición, generalización, realización y dependencia.

La forma importa

Rombo hueco o relleno, triángulo, línea continua o discontinua: cada detalle cambia el significado.

Referencia: OMG Unified Modeling Language (UML), versión 2.5.1 — Object Management Group.

